SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

Tiruchirappalli

CONTINUOUS LEARNING ASSESSMENT – 2

DSA – TASK 3

ALGORITHM BATTLE – WHICH DATA STRUCTURE WINS?

Project: Library Book Search Management System

Submitted by: _____

Registration Number: _____

Department / Semester: _____

Submitted to: _____

Academic Year 2026–2027

1. Introduction

The Library Book Search Management System is a simple application for storing and retrieving library book records. Each record contains a unique Book ID, Book Name, and Author. The same 30-record dataset is implemented using a Binary Search Tree (BST), an AVL Tree, and a hash table with Linear Probing. The purpose is to compare exact-key search, insertion behaviour, tree height, and hash collisions.

2. Problem Statement

A library catalogue must locate a book using its unique Book ID. A sequential scan can become slower as the number of records increases. This project compares three data structures that organise records differently and measure how many key comparisons or table probes are needed.

3. Objectives

• Implement insertion and searching in BST, AVL, and hashing.

• Display BST and AVL records using inorder traversal.

• Balance the AVL tree using LL, RR, LR, and RL rotations.

• Resolve hash collisions using Linear Probing.

• Use the same dataset and compare search effort, height, and collisions.

4. Dataset

The experiment uses 30 unique Book IDs: 101, 105, 110, 115, 120, 125, 130, 135, 140, 145, 150, 155, 160, 165, 170, 175, 180, 185, 190, 195, 200, 205, 210, 215, 220, 225, 230, 235, 240, 245. The same records and insertion order are used by all three programs. The IDs are inserted in ascending order.

Book ID	Book Name	Author
101	C Programming	Dennis Ritchie
105	Data Structures	Mark Allen
110	Operating Systems	Galvin
115	Computer Networks	Tanenbaum
120	DBMS	Navathe
125	Java Programming	Herbert Schildt
130	Python Basics	Mark Lutz
135	Computer Architecture	Morris Mano
140	Software Engineering	Pressman
145	Algorithms	Cormen
150	Web Technology	Jon Duckett
155	Artificial Intelligence	Russell
160	Machine Learning	Tom Mitchell
165	Cyber Security	William Stallings
170	Cloud Computing	Rajkumar Buyya
175	Data Mining	Han
180	Compiler Design	Aho
185	Computer Graphics	Hearn

Book ID	Book Name	Author
190	Digital Electronics	Morris Mano
195	Discrete Mathematics	Rosen
200	Linux Basics	Brian Ward
205	Database Systems	Elmasri
210	C++ Programming	Bjarne Stroustrup
215	Android Development	Reto Meier
220	Networking Basics	Kurose
225	Information Security	Whitman
230	Numerical Methods	Chapra
235	Theory of Computation	Sipser
240	Data Analytics	Foster Provost
245	Mobile Computing	Jochen Schiller

5. Binary Search Tree (BST)

A BST stores keys so that values smaller than a node are placed in its left subtree and larger values are placed in its right subtree. Search follows one branch according to key comparisons. Inorder traversal (Left, Root, Right) displays the records in ascending Book ID order.

Operations: insert, search, inorder traversal, and height calculation. The program counts one key comparison for each visited node. Height is measured in nodes on the longest root-to-leaf path; an empty tree has height zero.

Observation: Because the supplied IDs are inserted in ascending order, this particular BST becomes a right-skewed chain. This is an intentionally useful worst-case demonstration, not a claim that every BST always has this shape.

6. AVL Tree

An AVL tree is a self-balancing binary search tree. For each node, the balance factor is height(left subtree) minus height(right subtree). A node is balanced when its balance factor is -1, 0, or +1. After insertion, the program updates heights and applies the required rotation.

The four rotation cases are LL (single right rotation), RR (single left rotation), LR (left rotation on the left child followed by right rotation), and RL (right rotation on the right child followed by left rotation). Inorder traversal still displays records in ascending order.

7. Hashing with Linear Probing

The hash function is h(BookID) = BookID % 40. If the computed table slot is occupied by another key, Linear Probing checks the next slot, wrapping from the final slot to index zero. The program counts each occupied slot encountered during insertion as one collision. During search, each inspected slot is counted as one probe.

The table has 40 slots for 30 initial records (load factor 30/40 = 0.75). The hash table does not keep records in sorted order.

8. Experimental Method

All three programs receive the same 30 records in the same ascending insertion order. Before adding the extra record, run a successful search for Book ID 115 and an unsuccessful search for Book ID 999. Record tree heights for BST and AVL and initial insertion collisions for hashing. Finally, test insertion of Book ID 250, name DSA Advanced, author John Smith.

9. Experimental Results

Metric	BST	AVL	Hashing
Successful search for 115	4 comparisons	3 comparisons	1 probe(s)
Unsuccessful search for 999	30 comparisons	5 comparisons	1 probe(s)
Height (nodes)	30	5	Not applicable
Initial insertion collisions	Not applicable	Not applicable	41

These values were obtained by compiling and running the included C programs with GCC using the stated test keys and initial dataset. Re-run the programs on the student's own system and retain the console output as evidence. Search measurements are taken before the extra ID 250 insertion.

10. Analysis and Technical Comparison

BST is straightforward, but its performance depends on insertion order. Here, ascending insertion creates a chain of 30 nodes, so the successful search for 115 visits four nodes and the unsuccessful search for 999 visits all 30 nodes. AVL rotations maintain balance, giving a much smaller height and fewer comparisons for these tests. Hashing requires table probes rather than tree comparisons; for this hash function and table, the tested existing and missing keys are located with one probe each. The insertion collision count reflects the chosen hash function, table size, keys, and insertion order.

11. Complexity Comparison

Structure	Average search	Worst-case search	Ordered traversal
BST	O(log n)	O(n)	Yes, inorder
AVL	O(log n)	O(log n)	Yes, inorder
Hashing	O(1) average	O(n)	No

12. Final Recommendation

For this application's primary requirement—exact lookup by unique Book ID—hashing with Linear Probing is the recommended choice when the table is sized appropriately and the load factor is controlled. It provides constant-time average lookup under good hashing conditions. If the library also needs records maintained in sorted order or frequent range queries, AVL is a strong alternative because it guarantees logarithmic search and supports inorder traversal. A plain BST is simplest but is vulnerable to poor insertion order.

13. Conclusion

The experiment demonstrates that data-structure performance depends on organisation, balancing, hash function, table size, and input order. The sorted input makes the BST reach its worst-case shape. AVL avoids this through rotations. Hashing performs direct table lookup with Linear Probing to resolve collisions. The selected structure should match the application's dominant operations.

14. Files and Reproducibility

The accompanying project folder contains bst.c, avl.c, hashing.c, dataset.csv, dataset.txt, results.txt, sample_test_outputs.txt, and README.md. Compile each program with GCC. The results file records the exact test conditions and measured output.

15. Student Completion Before Submission

■ Fill in your name, registration number, department, semester, and faculty name on the cover page.

■ Read and understand all three programs; be prepared to explain insertion, search, rotations, hash function, and Linear Probing.

■ Run all three programs on your own computer and save screenshots or console output if required by faculty.

■ Check the course's AI-use policy and disclose assistance if required.

■ Upload source files, dataset, README, results, and final PDF to GitHub; submit the repository URL and PDF in Google Classroom.